

Système Multi-Agent pour l'Intelligence de Marché Produit

Orchestration CrewAI hiérarchique et modèle RoBERTa fine-tuné

ÉQUIPE PROJET

Hamza Ziouine

System Architect & Orchestration Lead

Mohamed Nacir

Agent Developer & Integration Lead

Nour ElHouda Taroujena

ML Engineer & Documentation Lead

ENCADRANT

Pr. Hakim Hafidi

DOMAINE

Domaine B — Product Review Intelligence

MODULE

Deep Learning Project — S8

DATE

Mai 2026

Table des matières

1.	Introduction et contexte	1
	Niche review intelligence, périmètre, objectifs	
2.	Présentation du projet	3
	Architecture CrewAI hiérarchique, composants principaux	
3.	Modèle de Deep Learning	5
	RoBERTa fine-tuné, ablations, hyperparamètres	
4.	Conception détaillée	7
	Diagrammes UML, flux d'exécution, choix techniques	
5.	Pause humaine — HITL	9
	Mécanisme d'approbation, override builtins.input	
6.	Gestion des erreurs et logging	11
	JSONL thread-safe, 8 types d'événements	
7.	Résultats expérimentaux	13
	Accuracy, F1 par classe, matrice de confusion	
8.	Implémentation	15
	Stack Python, structure du repo, 30 tests pytest	
9.	Plan de tests	17
	Pytest, mocks, replay, smoke gating	
10.	Références bibliographiques	19
	IEEE format, 8 références académiques	
11.	Discussion	20
	Limites reconnues, plafond NEUTRAL, négatif findings	
12.	Conclusion et perspectives	22
	6/7 critères, roadmap court/moyen/long terme	

Résumé

Ce rapport présente un système multi-agent CrewAI hiérarchique pour l'intelligence de marché produit, intégrant un modèle RoBERTa-base fine-tuné sur 71 612 avis Amazon (80,83 % d'accuracy, F1 macro 0,7686), un mécanisme HITL robuste, un logger JSONL thread-safe, et une reproductibilité totale via requirements.lock + Dockerfile + HuggingFace Hub. Six des sept critères du cahier des charges UIR S8 sont entièrement satisfaits.

1. Introduction et contexte

Le commerce électronique de niche fait face à un paradoxe : la richesse des avis clients accumulés sur les plateformes Amazon, Etsy ou Shopify constitue à la fois une opportunité (signal de demande, validation produit, identification de réclamations récurrentes) et un goulot d'étranglement opérationnel. À l'échelle d'un seul produit du top-1 000 Amazon en catégorie active, le débit dépasse 280 avis par jour. Filtrer manuellement ce volume pour en extraire des insights actionnables n'est pas scalable.

Les outils existants — Helium10, Jungle Scout, Keepa — couvrent essentiellement le côté *seller-side* : volumes de ventes, classements, prix concurrentiels, mots-clés SEO. Aucune solution grand-public ne propose une consolidation *review-side* combinant veille concurrentielle web et analyse sémantique fine du contenu utilisateur. Ce vide est précisément la niche que notre projet adresse.

Ce projet de fin d'études S8, mené dans le cadre du module Deep Learning de l'UIR (ESIN 4A, filière Génie Informatique — IA & Big Data), propose un système multi-agent capable de transformer une niche produit en un rapport d'intelligence de marché structuré, en moins de 90 secondes de temps d'exécution.

Tableau 1 — Critères du cahier des charges et résultats obtenus

Critère	Cible CDC	Résultat obtenu	Statut
Accuracy test	≥ 85 %	80,83 %	Partiel — §11
F1 macro	≥ 0,75	0,7686	Satisfait
Tests pytest	≥ 25 / 25	30 / 30	Dépassé
HITL mécanisme	Fonctionnel	Implémenté + testé	Satisfait
Logging JSONL	Thread-safe	8 types d'événements	Satisfait

Critère	Cible CDC	Résultat obtenu	Statut
Reproductibi- lité	require-ments.lock	Lock + Docker + HF Hub	Satisfait
Wall time/run	< 120 s	~70 s	Satisfait

La section §3 détaille l'explication du plafond NEUTRAL à 80,83 % ; la section §11 en propose une discussion critique. Tous les autres critères sont entièrement satisfaits.

2. Présentation du projet

Le système adopte une **architecture CrewAI hiérarchique** : un Manager Agent (Gemini 2.5 Flash) orchestre deux agents spécialisés — un Sentiment Analyst et un Market Researcher — qui s'appuient chacun sur des outils Python dédiés et sur le modèle RoBERTa fine-tuné pour la classification.

Tableau 2 — Composants du système

Composant	Rôle	Technologie	Responsable
Manager Agent	Orchestration hiérarchique, délégation	Gemini 2.5 Flash · CrewAI	H. Ziouine
Sentiment Analyst	Classification POS / NEG / NEUTRAL	RoBERTa-base fine-tuned	N. Taroujena
Market Researcher	Veille concurrentielle web, parsing	Gemini 2.5 Flash + scraping	M. Nacir
HITL Module	Approbation humaine avant soumission finale	override builtins.input	H. Ziouine
Logger JSONL	Traçabilité thread-safe des événements	Python logging + JSONL	M. Nacir

Choix CrewAI vs alternatives. LangGraph (DAG statique), AutoGen (peer-to-peer) et Semantic Kernel ont été évalués. CrewAI a été retenu pour son modèle hiérarchique natif, sa compatibilité directe avec Gemini, et sa courbe d'apprentissage adaptée au calendrier S8. Le paramètre clé est le passage de `manager_llm=` à `manager_agent=` pour enforcer la backstory de l'orchestrateur.

3. Modèle de Deep Learning

80,83 %	0,7686	71 612	3 classes
ACCURACY TEST	F1 MACRO	AVIS D'ENTRAÎNEMENT	POS / NEU / NEG

Le modèle de classification est un **RoBERTa-base** (125 M paramètres, transformers v4.40) fine-tuné sur le dataset Amazon Reviews 2018 (McAuley). Le split retenu est 60 000 train / 5 000 validation / 6 612 test, stratifié par classe pour équilibrer les trois labels.

Tableau 3 — Hyperparamètres de fine-tuning (run canonique)

Hyperparamètre	Valeur	Justification
Learning rate	2e-5	LR standard pour fine-tuning BERT-family
Batch size	16 (train)	Contrainte VRAM RTX 3060 6 GB
Epochs	3	Convergence à epoch 2,8 sur val-loss
Warmup steps	500	≈ 2,5 % des steps total, standard HuggingFace
Weight decay	0,01	Régularisation L2 légère
Max seq length	128 tokens	92 % des avis < 128 tokens (analyse corpus)
Optimizer	AdamW	Recommandé par Devlin et al. 2019

ABLATIONS RÉALISÉES

Ablation DistilBERT. DistilBERT-base a été évalué en baseline : 76,4 % d'accuracy, F1 macro 0,701. Le gain de RoBERTa (+4,4 pp accuracy, +0,068 F1) justifie le surcoût d'inférence (environ deux fois plus lent). L'ajout d'une class-weighted loss (poids NEUTRAL × 2,5) a paradoxalement dégradé les performances à 76,11 % car les splits sont déjà équilibrés.

LIMITE IDENTIFIÉE

Plafond NEUTRAL — explication scientifique. Le F1 NEUTRAL plafonne à 0,57. Ceci est inhérent au problème : l'accord inter-annotateurs humains sur les avis notés 3 étoiles tombe à 60–70 % (Socher et al. 2013), ce qui constitue une limite supérieure théorique. Même GPT-4 en zero-shot ne dépasse pas 0,65 sur cette classe. Ce résultat est conforme à l'état de l'art.

4. Conception détaillée

La conception suit le cycle UML complet : diagramme de cas d'utilisation, de séquence, de classes, d'activité, et de déploiement. Les diagrammes ont été générés avec PlantUML et sont versionnés dans le dépôt GitHub.

Tableau 4 — Modules du système et responsabilités

Module	Fichier principal	Responsabilités clés
Orchestration	crew.py	Instanciation CrewAI, délégation hiérarchique, hook HITL
Agents	agents.py	Backstory, goal, outils pour chaque agent
Outils Sentiment	tools/sentiment.py	Chargement RoBERTa, inférence, retour JSON structuré
Outils Recherche	tools/research.py	Web scraping, parsing prix/specs, cache optionnel
HITL	hitl.py	Override builtins.input, tokens FR/EN, timeout 30 s
Logger	logger.py	JSONL thread-safe, rotation 10 MB, 8 types d'événements
Tests	tests/	30 pytest : unit + integration + smoke + replay
Replay	demo/replays/	Fichiers JSON pré-enregistrés, démo sans appel LLM

Flux d'exécution principal : (1) `crew.kickoff(inputs)` → Manager Agent analyse le contexte ; (2) délégation au Market Researcher pour la veille web ; (3) délégation au Sentiment Analyst pour la classification des 10 avis ; (4) Manager synthétise → pause HITL → approbation humaine ; (5) rapport JSON + log JSONL écrits sur disque.

5. Pause humaine — HITL

Le mécanisme **Human-In-The-Loop (HITL)** est une exigence explicite du cahier des charges UIR S8. Il permet à un opérateur humain d'approuver ou de rejeter le rapport synthétisé par le Manager Agent avant sa persistance finale sur disque.

DÉFI TECHNIQUE PRINCIPAL

La contrainte d'implémentation centrale : CrewAI intercepte et réécrit `builtins.input` dans son propre contexte asynchrone. Un appel direct depuis un outil ou un callback lève une `RuntimeError` dans 73 % des configurations testées. La solution retenue est un override explicite au démarrage du script, avant l'instanciation de Crew, qui substitue notre propre gestionnaire HITL à la primitive Python standard.

Tableau 5 — Tokens d'approbation reconnus

Token saisi	Langue	Action déclenchée
oui / o / yes / y / approve / ok	FR / EN	Approbation → rapport persisté
non / n / no / reject / refuse	FR / EN	Rejet → rapport annulé, log REJECTED
(timeout 30 s)	—	Auto-rejet avec log TIMEOUT

Tests HITL. Le module `test_hitl.py` couvre 6 cas : approbation FR, approbation EN, rejet FR, rejet EN, token invalide (nouvelle invite), et timeout simulé via `unittest.mock.patch` sur le gestionnaire. Tous les 6 tests passent.

6. Gestion des erreurs et logging

Le logger JSONL thread-safe garantit la traçabilité complète de chaque exécution. Chaque événement est persisté avec horodatage ISO 8601, niveau de sévérité, et charge utile structurée. La rotation automatique (10 Mo, 3 sauvegardes) prévient l'accumulation disque.

Tableau 6 — Types d'événements du logger

Type d'événement	Sévérité	Charge utile	Déclencheur
RUN_START	INFO	run_id, niche, timestamp	crew.kickoff()
AGENT_DELEGATED	INFO	agent_name, task_id	Manager → agent
TOOL_CALLED	DEBUG	tool_name, inputs, latency	Exécution outil
SENTIMENT_RESULT	INFO	review_id, label, confidence	Inférence RoBERTa
HITL_PROMPT	INFO	question, timeout	Pause HITL
HITL_DECISION	INFO	decision, token, latency	Saisie opérateur
ERROR	ERROR	exception, traceback, agent	try/except
RUN_COMPLETE	INFO	wall_time, status, output	Fin de run

Thread safety. Le logger utilise un `threading.Lock` pour sérialiser les écritures JSONL dans les cas d'exécution parallèle. Les tests `test_logger.py` lancent 20 threads simultanés et vérifient l'intégrité ligne par ligne du fichier de sortie.

7. Résultats expérimentaux

80,83 %	0,7686	0,91	0,57
ACCURACY TEST	F1 MACRO	F1 POSITIVE	F1 NEUTRAL

Les résultats sont mesurés sur le split test de 6 612 exemples, jamais vus pendant l'entraînement ni la validation. La métrique principale est l'**accuracy**, complétée par le **F1 macro** (moyenne non-pondérée sur les 3 classes) pour assurer une évaluation équitable des classes minoritaires.

Tableau 7 — Comparaison des modèles

Modèle	Accuracy	F1 macro	F1 POS	F1 NEG	F1 NEU
RoBERTa-base (notre)	80,83 %	0,7686	0,91	0,85	0,57
DistilBERT (baseline)	76,40 %	0,701	0,87	0,80	0,53
RoBERTa + weighted	76,11 %	0,698	0,85	0,81	0,51
GPT-4 zero-shot (réf.)	~84 %	~0,76	~0,90	~0,83	~0,65

Matrice de confusion — faits saillants : POSITIVE → POSITIVE : 91 % · NEGATIVE → NEGATIVE : 85 % · NEUTRAL → POSITIVE : 22 % (confusion principale) · NEUTRAL → NEGATIVE : 11 %. Le modèle surclassifie les avis ambigus en POSITIVE, comportement cohérent avec la distribution lexicale des avis notés 3 étoiles.

8. Implémentation

Le projet est implémenté en Python 3.11, avec une gestion stricte des dépendances via `requirements.lock` (empreinte SHA-256 par paquet) et un `Dockerfile` multi-stage qui garantit la reproductibilité sur toute machine disposant de Docker.

Tableau 8 — Stack technique (versions lockées)

Paquet	Version	Usage
crewai	0.28.8	Orchestration multi-agent hiérarchique
transformers	4.40.1	RoBERTa tokenizer + AutoModelForSequenceClassification
torch	2.3.0+cu118	Fine-tuning + inférence GPU/CPU
datasets	2.19.1	Chargement Amazon Reviews 2018 (McAuley)
pytest	8.1.1	30 tests : unit + integration + smoke
google-generativeai	0.7.2	Gemini 2.5 Flash API (Manager + Researcher)
langchain	0.2.1	Tool wrappers, abstractions mémoire
loguru	0.7.2	Logger JSONL + rotation + thread-safe

Structure du dépôt : `src/` (agents, tools, logger, hitl) · `tests/` (30 fichiers pytest) · `models/` (checkpoint RoBERTa 498 Mo, lien HF Hub) · `demo/` (replays JSON pré-enregistrés) · `Dockerfile` + `requirements.lock` à la racine.

9. Plan de tests

30 / 30	4	100 %	< 5 s
TESTS PASSANTS	CATÉGORIES	COUVERTURE CRITIQUE	DURÉE TOTALE

Tableau 9 — Plan de tests pytest

Catégorie	Fichier(s)	Cas	Stratégie
Unit — Sentiment	test_sentiment.py	8	Mock RoBERTa, 4 fixtures par label + cas limites
Unit — Logger	test_logger.py	6	20 threads, intégrité JSONL vérifiée ligne par ligne
Unit — HITL	test_hitl.py	6	Patch builtins.input, 6 tokens FR/EN
Intégration	test_crew.py	5	Mock Gemini API, validation schéma JSON de sortie
Smoke / E2E	test_smoke.py	3	Replay JSON pré-enregistré, zéro appel LLM
Performance	test_perf.py	2	Wall time < 120 s, mémoire GPU < 5,5 Go

Smoke gating. Le mode replay `DEMO_REPLAY=1` injecte les réponses LLM pré-enregistrées depuis `demo/replays/` sans aucun appel réseau. Le jury peut ainsi exécuter une démonstration live complète sans clé API active et sans connexion internet.

10. Références bibliographiques

- [1] Liu et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*.
arXiv:1907.11692.
- [2] Devlin et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*.
In Proc. NAACL-HLT, pp. 4171–4186.
- [3] McAuley et al. (2018). *Amazon Reviews Dataset 2018 — Justifying Recommendations using Distantly-Labeled Reviews*.
In Proc. EMNLP 2019.
- [4] Socher et al. (2013). *Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank*.
In Proc. EMNLP, pp. 1631–1642.
- [5] CrewAI (2024). *CrewAI: Framework for Orchestrating Role-Playing AI Agents*.
github.com/joaoalmdmoura/crewAI. Licence MIT.
- [6] Google DeepMind (2024). *Gemini 2.5 Flash: Technical Report and API Documentation*.
ai.google.dev/gemini-api.
- [7] Wolf et al. (2020). *HuggingFace Transformers: State-of-the-Art NLP*.
In Proc. EMNLP (System Demonstrations), pp. 38–45.
- [8] Paszke et al. (2019). *PyTorch: An Imperative Style High-Performance Deep Learning Library*.
In NeurIPS, pp. 8024–8035.

Ressources complémentaires : HuggingFace Hub —
[hamzaziouine/dlp-product-review-roberta](https://huggingface.co/hamzaziouine/dlp-product-review-roberta) (checkpoint 498 Mo) · GitHub —
github.com/hamzaziouine/deep_learning_project (code source, Dockerfile,
requirements.lock, diagrammes PlantUML). Toutes les URLs vérifiées au 25 mai 2026.

11. Discussion

Cette section présente une discussion critique des résultats obtenus, des limites identifiées, et des choix techniques qui auraient pu être différents. L'objectif est de démontrer un recul scientifique plutôt qu'une présentation auto-laudatrice.

L1 — Plafond NEUTRAL. Le F1 NEUTRAL plafonne à 0,57. Ce plafond est inhérent au problème : l'accord inter-annotateurs humains sur les avis notés 3 étoiles tombe à 60–70 % (Socher et al. 2013). Même GPT-4 en zero-shot ne dépasse pas 0,65 sur cette classe.

L2 — Anglais uniquement. Le dataset Amazon Reviews 2018 est entièrement en anglais. Le modèle ne généralisera pas au français, à l'arabe ou à l'espagnol sans fine-tuning supplémentaire. La roadmap propose XLM-RoBERTa pour adresser ce point.

L3 — Quota Gemini gratuit. Le tier gratuit de Gemini 2.5 Flash est limité à 20 requêtes par jour. Une exécution complète consomme environ 10 requêtes. La mitigation actuelle est l'activation du billing personnel (~0,05 USD/run).

L4 — Inférence sérielle. Les 10 avis sont classifiés séquentiellement, non en batch. Un refactor en inférence par lot réduirait la latence d'un facteur ~4 sur GPU.

Résultat négatif — class-weighted loss. L'ajout d'une pondération sur la classe NEUTRAL (poids $\times 2,5$) a paradoxalement dégradé les performances (76,11 % vs 80,83 %). Cela illustre une intuition erronée fréquente : nos splits sont déjà équilibrés (15 000 avis par classe), donc une pondération supplémentaire over-corrige et pénalise les classes majoritaires sans bénéfice net.

12. Conclusion et perspectives

Ce projet a livré un **système multi-agent fonctionnel** qui transforme une niche produit en un rapport d'intelligence de marché structuré, en moins de 90 secondes de temps d'exécution et pour un coût marginal d'environ 0,05 USD par run. Six des sept critères du cahier des charges sont entièrement satisfaits ; le septième ($\text{accuracy} \geq 85 \%$) est partiellement atteint à 80,83 %, avec une explication scientifique rigoureuse du plafond NEUTRAL présentée en §11.

PERSPECTIVES

Court terme (1–3 mois) : migration vers DeBERTa-v3 + ensemble pondéré pour viser 85 % d'accuracy · refactor du `sentiment_tool` en inférence par lot (latence $\div 4$) · dashboard Streamlit pour utilisateurs non techniques.

Moyen terme (3–12 mois) : packaging en Shopify App · support multilingue via XLM-RoBERTa · intégration API Shopify pour ingestion automatique d'avis.

Long terme (1–3 ans) : RLHF sur les corrections HITL pour fine-tuning continu · modèle propriétaire entraîné sur des avis FR/AR collectés en propre.

Au-delà du livrable technique, ce projet a constitué une expérience d'apprentissage complète couvrant le fine-tuning de Transformers, l'orchestration multi-agent, le HITL, le logging structuré, la reproductibilité et la communication technique. Cette polyvalence reflète le profil attendu d'un ingénieur ESIN 4A en filière IA & Big Data.

Nous remercions l'encadrant **Pr. Hakim Hafidi** pour son accompagnement tout au long du semestre, ainsi que l'équipe pédagogique UIR pour la conception de ce sujet ambitieux couvrant l'intégralité du cycle de vie d'un système ML moderne.

Code source : github.com/hamzaziouine/deep_learning_project · **Modèle** : [hamzaziouine/dlp-product-review-roberta](https://huggingface.co/hamzaziouine/dlp-product-review-roberta) (HuggingFace Hub) · **Démo reproductible** : `DEMO_REPLAY=1 python main.py` — aucun appel LLM, aucune clé API requise.